

Diagnostic Analysis and Remediation Framework for Bindless Texture Sampling Failures in Offscreen Vulkan Render Passes

A critical blocker in modern graphics hardware interfaces utilizing Vulkan dynamic rendering and bindless descriptor architectures occurs when offscreen color attachments fail to sample correctly in subsequent render passes¹. During the implementation of an offscreen Dual-Kawase blur algorithm within a Rust-based Render Hardware Interface (RHI), an issue was identified where sampling a bindless texture while rendering into an offscreen target

consistently returns all-zero data $(0.0, 0.0, 0.0, 0.0)$ ¹. This behavior persists on hardware without triggering Vulkan validation layer errors, contrasting sharply with swapchain rendering passes where the identical bindless sampling path functions as intended¹.

This report provides a systematic diagnostic breakdown of the failure mode, synthesizes empirical findings across twelve eliminated hypotheses, establishes Vulkan specification-level root cause mechanisms, and outlines technical remediation procedures for the RHI engine architecture¹.

1. Architectural Context and Failure Symptomology

The failure manifests during multi-pass offscreen filtering chains, specifically during the execution of a Dual-Kawase downsample pass¹. The RHI renders initial scene content into a transient offscreen color attachment (L_0), transitions L_0 to a bindless descriptor slot, and then executes a downsample pass by sampling L_0 through the bindless descriptor array while rendering into a second offscreen transient target (L_1)¹.

During execution, GPU pixel readbacks demonstrate that the shader reads zero values for every texel query across L_0 , causing the resulting pass L_1 to output background clear colors across the target¹.

Empirical Pipeline Characteristics

- **Swapchain Isolation Paradox:** Sampling the exact same bindless texture slot while rendering directly into the swapchain backbuffer produces correct visual output¹. The failure condition is uniquely triggered when the active color render target is an offscreen image target reached via `begin_render_to_texture` or `begin_render_to_texture_no_end`¹.
- **Pipeline and Shader Integrity:** Substituting shader output with hardcoded constants confirms that viewport transformation, scissor testing, rasterization, pipeline binding, push constants, vertex streams, and color attachment blend states function correctly¹.

- **Input Parameter Propagation:** Directly outputting interpolated texture coordinates (`frag_uv`) and push constant texture indices (`pc.texture_index`) as color channels confirms that coordinates and indices reach the fragment shader without numerical corruption or truncation¹.
- **Validation Layer Silence:** Standard Vulkan Validation Layers return zero warnings or errors during command buffer recording, queue submission, or queue execution¹.

During the initial investigation, a secondary issue involving double `vkCmdEndRendering` command buffer calls was identified when directly chaining `end_render_to_texture` into `begin_render_to_texture`¹. This issue was resolved by introducing `RhiCommandBuffer::begin_render_to_texture_no_end`, ensuring that render pass end calls are decoupled from barrier insertions and subsequent pass initialization¹. While necessary for driver compliance, resolving this double-end error did not alter the all-zero sampling behavior¹.

2. Empirical Hypothesis Elimination Matrix

An extensive diagnostic elimination process was executed to isolate the failure mechanism, testing twelve independent variables across the host RHI and GPU execution state¹.

Hypothesis ID	Investigated Variable	Diagnostic Methodology	Observed Result	Elimination Conclusion
H1	Redundant <code>cmd_end_rendering</code> Calls	Introduced <code>begin_render_to_texture_no_end</code> to prevent double-ending active passes ¹ .	Validation error eliminated; sampling failure persisted ¹ .	Independent bug resolved; not the root cause of zero reads ¹ .
H2	Mid-Pass Bindless Registration	Moved <code>register_bindless</code> outside active render passes ¹ .	Execution order aligned with swapchain pass structures ¹ .	Registration timing relative to pass boundaries ruled out ¹ .
H3	Loss of nonuniformEXT Specifier	Fully inlined descriptor indices at every sampling tap in GLSL ¹ .	Layout matched working reference shaders byte-for-byte ¹ .	Decorator stripping in GLSL compilation ruled out ¹ .

H4	Multi-Tap Filter Offset Math	Reduced multi-tap Kawase kernel to single unweighted texture() call ¹ .	Shader reads returned zero across all UV coordinates ¹ .	Sample offset math and kernel scaling ruled out ¹ .
H5	Descriptor Slot Overwrite	Maintained distinct descriptor indices (<i>Slot₀ ≠</i>) across frames ¹ .	Verified slot separation (0 vs 1) in host tracking ¹ .	Descriptor index collision or write race ruled out ¹ .
H6	Render Target Format Mismatch	Evaluated floating-point (Rgba16Float) against Bgra8Srgb swapchain formats ¹ .	Both formats failed identically during offscreen passes ¹ .	Format-specific sampler incompatibility ruled out ¹ .
H7	Render Target Scale Delta	Tested 1:1 matching dimensions versus 1:2 downsample resolutions ¹ .	Both target size configurations failed identically ¹ .	Scale mismatch or mipmap generation errors ruled out ¹ .
H8	Shader Source Code Defects	Replaced Kawase fragment shader with unmodified, proven bindless_textured.frag ¹ .	Proven shader failed when targeted offscreen ¹ .	Shader source code or compilation logic ruled out ¹ .
H9	Push Constant / UV	Visualized frag_uv and pc.texture_ind	Colors displayed expected	Pipeline input assembly and stage routing

	Corruption	ex directly as RGB output channels ¹ .	gradients and index values ¹ .	ruled out ¹ .
H10	Missing Device Feature Flags	Audited descriptor indexing features (runtime_descriptor_array, partially_bound) ¹ .	All required device features confirmed active ¹ .	Device initialization or missing extensions ruled out ¹ .
H11	Pipeline / Render Pass Setup	Emitted hardcoded <code>vec4(1.0, 0.0, 0.0, 1.0)</code> color output ¹ .	Red pixels rasterized successfully across composite targets ¹ .	Rasterization, blending, viewport, and scissor ruled out ¹ .
H12	Render Target Context	Tested identical bindless sampling calls against swapchain versus offscreen targets ¹ .	Swapchain succeeds; offscreen target reads zero ¹ .	Primary root cause dependency established ¹ .

3. Analysis of Unresolved Root Cause Candidates

Because basic pipeline parameters, GLSL shader source code, and device feature flags have been ruled out, the failure points to specific interactions within the Vulkan specification state machine¹. Four technical mechanisms explain why bindless texture sampling returns zero exclusively during offscreen render passes.

3.1. Pipeline Barrier Memory Coherency and Layout Synchronization

When transitioning an offscreen image (L_0) from a color attachment output to a fragment shader input for pass L_1 , explicit memory barriers and pipeline stage dependencies must be recorded. Swapchain backbuffers mask missing barriers because presentation engines force synchronization during image acquisition and queue submission. Offscreen transient targets lack presentation boundaries.

If the RHI omits an explicit `VkImageMemoryBarrier2` between the end of pass L_0 and the start of pass L_1 , two hardware-level failures occur. First, the GPU L2 cache holds the written color attachment data, but the L1 texture cache utilized by the Execution Units has not been invalidated, causing texture units to read stale memory segments containing zero. Second, if L_0 remains in `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` without transitioning to `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`, hardware memory compression units (such as AMD DCC or Nvidia Intel Compressed Formats) do not decompress color blocks prior to sampling, resulting in zeroed reads.

3.2. Descriptor Set Compatibility and Binding Scope Under Dynamic Rendering

Under Vulkan Dynamic Rendering (`VK_KHR_dynamic_rendering`), descriptor sets bound prior to `vkCmdBeginRendering` remain valid only if the pipeline layout bound during the offscreen pass is identical to or layout-compatible with the pipeline layout used when binding the descriptor set.

If the offscreen rendering pipeline uses a distinct `VkPipelineLayout` handle—or if the RHI executes `vkCmdBindPipeline` with a layout that redefines Set 0 or Push Constant ranges—Vulkan invalidates previously bound descriptor sets for that set index. Under Vulkan's `descriptorBindingPartiallyBound` feature, sampling an invalidated descriptor slot does not

trigger a driver crash or validation layer error; instead, the hardware returns 0.0 for all channels.

3.3. Descriptor Set Layout Update-After-Bind Flags

When registering bindless textures dynamically via host writes (`vkUpdateDescriptorSets`), the underlying descriptor pool and descriptor set layout must explicitly configure `VK_DESCRIPTOR_SET_LAYOUT_CREATE_UPDATE_AFTER_BIND_POOL_BIT` and `VK_DESCRIPTOR_BINDING_UPDATE_AFTER_BIND_BIT`. If `register_bindless` updates the descriptor array on the host while command buffers are being recorded or executed, but the descriptor set layout was instantiated without update-after-bind flags, host updates execute without raising a validation error while deferring GPU descriptor page updates until device idle.

Execution units sampling L_0 's descriptor index consequently read an unpopulated page entry.

3.4. Offscreen Image Creation Flags and Sampler Component Swizzle

Offscreen images constructed for transient passes must explicitly declare sampling usage flags during creation via `VkImageCreateInfo`, requiring `VK_IMAGE_USAGE_SAMPLED_BIT` alongside `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`. Omitted sampling flags allow drivers to place image memory in non-sampled memory banks. Furthermore, if `VkImageViewCreateInfo.components` is populated with `VK_COMPONENT_SWIZZLE_ZERO` for any channel during offscreen target initialization, the GPU texture sampler overrides fetched texel channels with absolute zeros regardless of underlying memory content.

4. Systematic Diagnostic Protocol

To isolate the precise root cause on physical hardware, graphics engineers should execute a diagnostic protocol combining Vulkan synchronization validation and frame capture analysis.

Frame Debugging Procedure

1. **Enable Extended Synchronization Validation:** Configure Vulkan validation layers to track pipeline synchronization by enabling `VK_VALIDATION_FEATURE_ENABLE_SYNCHRONIZATION_VALIDATION_EXT`. This identifies missing pipeline barriers between L_0 color writes (`VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT`) and L_1 fragment reads (`VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT`).
2. **Inspect Pipeline State in RenderDoc:** Capture a frame during the execution of `dual_kawase_blur_demo.rs`¹. Select the downsample draw call in pass L_1 and verify that Set 1 lists a valid `VkImageView` handle for index `pc.texture_index`. If the binding slot displays as unbound or null, the pipeline layout bound prior to the draw call is incompatible with the descriptor set layout.
3. **Inspect Subresource Image Layouts:** Verify within the RenderDoc Resource Inspector that image L_0 resides in `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` during pass L_1 . If it remains in `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`, the RHI barrier engine failed to dispatch an layout transition barrier.
4. **Evaluate Pixel History and Texel Fetch:** Open Pixel History on target L_1 and step through the fragment shader execution trace. Evaluate whether the GLSL `texture()` instruction returns zeros due to an unpopulated descriptor slot versus sampling uninitialized physical memory.

5. Technical Solutions and RHI Architecture Remediation

To resolve offscreen bindless texture sampling failures across Vulkan drivers, implement the following modifications within the Rust RHI implementation.

5.1. Explicit Pipeline Barriers for Transient Target Layout Transitions

Ensure that the transition from rendering into L_0 to sampling L_0 in L_1 executes an explicit pipeline barrier using `vkCmdPipelineBarrier2` via ash.

```
Rust
```

```
pub fn transition_attachment_to_shader_read
```

```

command_buffer vk::CommandBuffer
image vk::Image
{
    let image_memory_barrier = vk::ImageMemoryBarrier2::builder()
        .src_stage_mask(vk::PipelineStageFlags2::COLOR_ATTACHMENT_OUTPUT)
        .src_access_mask(vk::AccessFlags2::COLOR_ATTACHMENT_WRITE)
        .dst_stage_mask(vk::PipelineStageFlags2::FRAGMENT_SHADER)
        .dst_access_mask(vk::AccessFlags2::SHADER_READ)
        .old_layout(vk::ImageLayout::COLOR_ATTACHMENT_OPTIMAL)
        .new_layout(vk::ImageLayout::SHADER_READ_ONLY_OPTIMAL)
        .subresource_range(vk::ImageSubresourceRange
            .aspect_mask(vk::ImageAspectFlags::COLOR)
            .base_mip_level(0)
            .level_count(1)
            .base_array_layer(0)
            .layer_count(1))
        .image(image)
        .build()

    let dependency_info = vk::DependencyInfo::builder()
        .image_memory_barriers(std::slice::from_ref(&image_memory_barrier))
        .build()

    unsafe {
        device.cmd_pipeline_barrier2(command_buffer, &dependency_info)
    }
}

```

5.2. Enforcing Descriptor Binding Within Dynamic Passes

Ensure that descriptor sets are explicitly bound inside active offscreen passes after pipeline binding to guarantee state validity under dynamic rendering scope changes.

```

Rust
pub fn begin_offscreen_pass_and_bind_state(
    device: &ash::Device,
    cmd_buf: vk::CommandBuffer,
    pipeline: vk::Pipeline,
    pipeline_layout: vk::PipelineLayout,
    bindless_descriptor_set: vk::DescriptorSet,
) {
    device.cmd_bind_pipeline(cmd_buf, pipeline, pipeline_layout)
    device.cmd_bind_descriptor_sets(cmd_buf, vk::PipelineBindPoint::GRAPHICS, pipeline_layout, 0, 1, &bindless_descriptor_set)
}

```

```

    rendering_info: &vk::RenderingInfo
}

unsafe {
    device.cmd_begin_rendering(cmd_buf, rendering_info);
    device.cmd_bind_pipeline(cmd_buf, vk::PipelineBindPoint::GRAPHICS, pipeline);

    device.cmd_bind_descriptor_sets(
        cmd_buf,
        vk::PipelineBindPoint::GRAPHICS,
        pipeline_layout,
        1 // Bindless descriptor set index
        std::slice::from_ref(&bindless_descriptor_set),
        &|
    );
}

```

5.3. Correcting Transient Texture Creation Parameters

When allocating transient textures for offscreen targets, enforce explicit image usage flags and identity component mappings within the RHI memory manager.

```

Rust
pub fn create_transient_render_target(
    device: &ash::Device,
    width: u32,
    height: u32,
    format: vk::Format,
) -> (vk::Image, vk::ImageView) {
    let image_info = vk::ImageCreateInfo::builder()
        .image_type(vk::ImageType::TYPE_2D)
        .format(format)
        .extent(vk::Extent3D { width, height, depth: 1 })
        .mip_levels(1)
        .array_layers(1)
        .samples(vk::SampleCountFlags::TYPE_1)
        .tiling(vk::ImageTiling::OPTIMAL)
        .usage(vk::ImageUsageFlags::COLOR_ATTACHMENT
            | vk::ImageUsageFlags::SAMPLED)
        .sharing_mode(vk::SharingMode::EXCLUSIVE);
}

```



```
        .initial_layout(vk::ImageLayout::UNDEFINED)

let image = unsafe { device.create_image(&image_info, None, unwrap())

let view_info = vk::ImageViewCreateInfo::builder()
    .image(image)
    .view_type(vk::ImageViewType::TYPE_2D)
    .format(format)
    .components(vk::ComponentMapping {
        r: vk::ComponentSwizzle::IDENTITY,
        g: vk::ComponentSwizzle::IDENTITY,
        b: vk::ComponentSwizzle::IDENTITY,
        a: vk::ComponentSwizzle::IDENTITY,
    })
    .subresource_range(vk::ImageSubresourceRange {
        aspect_mask: vk::ImageAspectFlags::COLOR,
        base_mip_level: 0,
        level_count: 1,
        base_array_layer: 0,
        layer_count: 1,
    })

let view = unsafe { device.create_image_view(&view_info, None, unwrap())
    .image_view
}
```

6. Strategic Implementation Roadmap

To resolve Finding #130 and unblock the visual filter pipeline (Step 7.2.1 and Step 7.2.2), the RHI implementation must follow a four-phase remediation sequence¹.

Roadmap Phase	Core Objective	RHI Action Items	Success Verification
Phase 1: Synchronization Audit	Enforce explicit image memory barriers ¹ .	Integrate transition_attachment_to_shader_read between offscreen pass ends and subsequent pass begins ¹ .	Zero synchronization validation warnings during frame recording ¹ .

Phase 2: Layout Consolidation	Standardize pipeline layouts across render passes ¹ .	Unify VkPipelineLayout definitions across offscreen and swapchain pipelines; re-bind descriptor sets post begin_rendering ¹ .	Descriptor Set 1 remains bound and populated across offscreen passes ¹ .
Phase 3: Frame Capture Validation	Verify texel fetch integrity in RenderDoc ¹ .	Capture dual_kawase_blur_d emo.rs and inspect downsample texel reads ¹ .	Pixel debug output confirms non-zero color values fetched from L_0 ¹ .
Phase 4: CI Re-integration	Unblock Step 7.2.1 and Step 7.2.2 ¹ .	Re-enable pixel assertions in dual_kawase_blur_d emo.rs; add demo to ci.yml; proceed to layer blending implementation ¹ .	Automated CI pipeline passes pixel assertions across all hardware targets ¹ .